

COMP527 - CA Assignment 2 - Clustering

Pablo Ramos - 201885602

16th April 2026

1 Overview of the Pipeline

`clustering.py` calls the functions from `preprocessing.py` to call and clean the data and then uses TF-IDF vectorisation on the data to get a mathematical abstraction of the sentences. Latent Semantic Analysis was used to simplify the geometry of the resulting vector set. L2 normalisation was implemented as well.

This final set was then used to get the silhouette analysis.

2 Term Frequency - Inverse Document Frequency (TF-IDF)

TF-IDF is a statistical method that determines the relevance of a word in relation to the document it appears in. In this case, the term frequency for a single term t of a sentence s is described as:

$$TF = \frac{\text{Frequency of } t}{\text{Count of unique terms in } s}$$

The inverse document frequency is described mathematically as:

$$IDF = \log \left(\frac{\text{Total number of sentences}}{\text{Count of sentences with term } t} \right)$$

The final TF-IDF score for a term is the product of both values. Words that appear frequently in a sentence but rarely across the dataset score highly, making them strong signals for distinguishing clusters.

3 Latent Semantic Analysis (LSA)

TF-IDF produces a high-dimensional sparse matrix, which is problematic for clustering due to the curse of dimensionality — in high-dimensional spaces, distances between points tend to converge, making it harder to form well-separated clusters. LSA addresses this by applying Truncated Singular Value Decomposition (SVD) to reduce the matrix to a lower-dimensional dense representation, whilst preserving the most meaningful semantic relationships between terms.

After experimenting with several values, `n_components=10` was selected. Whilst this retains only 6.89% of the explained variance, it produced the highest silhouette scores and, upon inspection, yielded semantically coherent clusters.

4 L2 Normalisation

L2 normalisation was applied to the LSA-reduced matrix, scaling each sentence vector to unit length. This ensures that K-Means computes distances based on the direction of vectors rather than their magnitude, preventing longer sentences from dominating the clustering simply due to having more terms.

5 Silhouette Analysis

Silhouette analysis was used to determine the optimal number of clusters K . For each data point, the silhouette coefficient s is defined as:

$$s = \frac{b-a}{\max(a,b)}$$

where a is the mean intra-cluster distance and b is the mean distance to the nearest neighbouring cluster. The overall score for a given K is the average across all points, with values ranging from -1 (poor) to 1 (ideal). K-Means was run for $K = 1$ to 10 , with $K = 1$ hardcoded to a score of 0 as the coefficient is undefined for a single cluster.

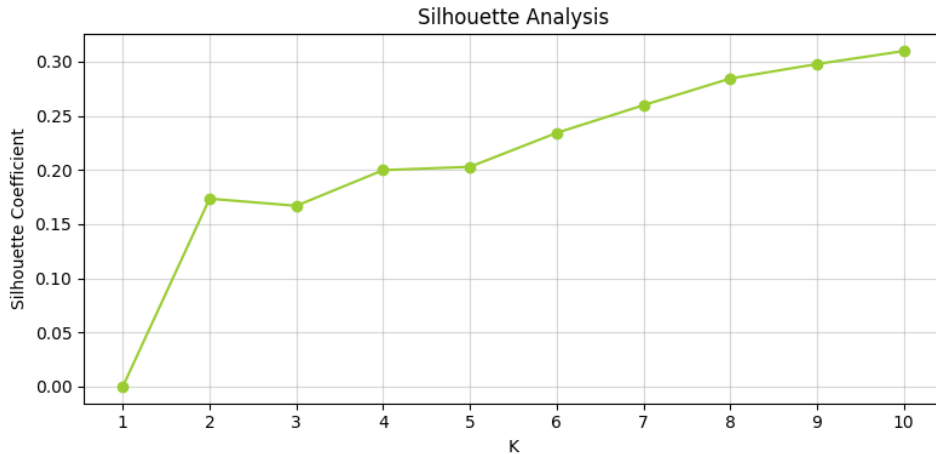


Figure 1: Silhouette coefficients for $K = 1$ to 10 .

6 Optimal K and Results

As shown in Figure 1, silhouette scores increase consistently with K , peaking at $K = 10$ with a score of 0.3097 . This was selected as the optimal value. Inspection of the resulting clusters revealed genuine semantic structure — Shakespeare dialogue was sub-clustered by character and scene, whilst reviews were grouped broadly by length and style. The relatively low absolute scores reflect the mixed nature of the dataset rather than a failure of the method.